

Préambule — Bien démarrer avec PHP

1) C'est quoi PHP ?

PHP est un langage **côté serveur** : le code PHP s'exécute **sur le serveur**, puis il génère du **HTML** (ou du JSON, etc.) envoyé au navigateur.

- **Navigateur (client)** : affiche du HTML/CSS/JS.
- **Serveur** : exécute le PHP et renvoie le résultat.

👉 Le navigateur **ne “voit” pas** le code PHP : il reçoit seulement le résultat (souvent du HTML).

2) Créer un fichier PHP

1. Ouvre un éditeur (VS Code conseillé).
2. Crée un fichier qui se termine par `.php`, par exemple :

- `index.php`
- `test.php`

✅ Un fichier PHP peut contenir :

- du **PHP**
- du **HTML**
- les deux mélangés (très courant au début)

Exemple minimal :

```
<?php
echo "Hello from PHP!";
```

3) Exécuter du PHP (très important)

Un fichier PHP ne fonctionne pas “juste” en double-cliquant dessus. Il doit être exécuté par un **serveur web avec PHP**.

Option A (classique) : XAMPP / MAMP / WAMP

1. Mets ton projet dans le dossier serveur :
 - XAMPP : `htdocs/mon_projet`
2. Lance Apache.
3. Ouvre :
 - `http://localhost/mon_projet/index.php`

Option B (simple en local) : serveur intégré PHP

Dans le terminal, dans le dossier du projet :

```
php -S localhost:8000
```

Puis dans le navigateur :

- `http://localhost:8000`

4) Les balises PHP

Le PHP s'écrit entre :

```
<?php
// code here
?>
```

- Si tu es en "PHP only", tu peux **souvent omettre** `?>` à la fin (bonne pratique).
- En mélange HTML + PHP, tu vas souvent ouvrir/fermer ces balises.

5) Afficher du texte : `echo`

Pour afficher quelque chose, on utilise `echo` :

```
<?php
echo "Hello!";
```

Dans une page HTML :

```
<h1><?php echo "welcome"; ?></h1>
```

Raccourci très utilisé :

```
<h1><?= "welcome" ?></h1>
```

`<?= ... ?>` est équivalent à `<?php echo ... ?>`.

Base de PHP : variables, types, opérateurs

6) Déclarer des variables

En PHP, une variable commence toujours par `$` :

```
<?php
$title = "Dune";
$year = 1965;
$price = 14.90;
$inStock = true;
```

Types courants (à connaître)

- `string` : texte `"Hello"`
- `int` : entier `42`
- `float` : nombre à virgule `19.90`
- `bool` : `true` / `false`
- `array` : tableau `[]`
- `null` : "vide / rien"

👉 PHP est "souple" : il peut changer de type, mais on garde de bonnes habitudes (clarté).

7) Concaténation (coller du texte)

On colle des morceaux de texte avec `.` :

```
<?php
$firstName = "Alex";
echo "Hello " . $firstName;
```

8) Opérateurs utiles

- Addition : `+`
- Soustraction : `-`
- Multiplication : `*`
- Division : `/`

Comparaisons :

- `==` égal (compare valeur, peut convertir)
- `===` strict égal (valeur **et** type) ✅ recommandé
- `!=` différent
- `>` `<` `>=` `<=`

Logique :

- `&&` ET
- `||` OU
- `!` NON

Exemple :

```
<?php
$age = 17;
if ($age >= 18) {
    echo "Adult";
} else {
    echo "Minor";
}
```

Conditions, boucles, tableaux, fonctions

9) Conditions `if` / `elseif` / `else`

```
<?php
$score = 75;

if ($score >= 90) {
    echo "A";
} elseif ($score >= 80) {
    echo "B";
} elseif ($score >= 70) {
    echo "C";
} else {
    echo "D";
}
```

10) Boucles

`for` (quand on a un compteur)

```
<?php
for ($i = 1; $i <= 5; $i++) {
    echo $i;
}
```

`foreach` (très utilisé avec les tableaux)

```
<?php
$books = ["Dune", "1984", "The Hobbit"];

foreach ($books as $book) {
    echo $book . "<br>";
}
```

11) Tableaux (arrays)

Tableau simple (indexé)

```
<?php
$books = ["Dune", "1984"];
echo $books[0]; // "Dune"
```

Tableau associatif (clé => valeur)

```
<?php
$book = [
    "title" => "Dune",
    "year" => 1965
];

echo $book["title"];
```

Tableau de tableaux (liste d'objets)

```
<?php
$library = [
    ["title" => "Dune", "year" => 1965],
    ["title" => "1984", "year" => 1949],
];
```

Objet

```
$book = (object)[
    "title" => "Dune",
    "year" => 1965
];

echo $book->title; // accès objet
```

Convertir un tableau en objet

Conversion simple (1 niveau)

```
$array = ["name" => "Alex", "age" => 17];
$obj = (object)$array;

echo $obj->name; // Alex
```

⚠ Attention : si tu as des tableaux imbriqués, la conversion simple ne convertit pas tout en profondeur.

3) Objets “simples” sans classes (stdClass)

PHP utilise souvent `stdClass` comme objet générique.

```
$book = new stdClass();  
$book->title = "Dune";  
$book->author = "Frank Herbert";  
  
echo $book->author;
```

C'est très bien pour comprendre `->` sans encore introduire les classes.

Liste d'objets (comme une “table” de données)

Tu peux avoir un tableau qui contient plusieurs objets :

```
$library = [  
    (object)["title" => "Dune", "year" => 1965],  
    (object)["title" => "1984", "year" => 1949],  
];  
  
foreach ($library as $book) {  
    echo $book->title . "<br>";  
}
```

Ici :

- `$library` = tableau (liste)
- `$book` = objet (un élément)

12) Fonctions

Une fonction sert à **réutiliser** du code :

```
<?php  
function formatPrice(float $price): string {  
    return "CHF " . number_format($price, 2);  
}  
  
echo formatPrice(14.9);
```

13) Bonnes pratiques dès le début (important pour les apprentis)

Sécuriser l'affichage HTML : `htmlspecialchars`

Si tu affiches du texte venant d'un utilisateur, il faut l'échapper :

```
<?= htmlspecialchars($title, ENT_QUOTES, "UTF-8") ?>
```

👉 Ça évite des problèmes de sécurité (XSS) et des bugs d'affichage.

Indentation + noms clairs

- Variables en anglais : `$bookTitle`, `$authorName`
- Fonctions en anglais : `validateBook()`
- Code lisible > code court

Mini modèle “page PHP classique”

Ce modèle est pratique pour enseigner :

```
<?php
// 1) Prepare data
$title = "Dune";
?>
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Demo</title>
</head>
<body>
  <!-- 2) Display -->
  <h1><?= htmlspecialchars($title) ?></h1>
</body>
</html>
```